

FINAL INTERNATIONAL UNIVERSITY

Faculty of Engineering

Computer Engineering Program

LAB Manual 3

Course Title: CMPE341- Database Systems

Prepared by (LAB Instructor)

Sen. Inst. Imane BOUMEDRA

Fall-2026-2027

Library Management System: ER Diagram to MySQL (DDL & DML)

Student name: _____ Student number: _____ Date: _____

Duration: 2–3 hours | Software: diagrams.net (draw.io) and MySQL Workbench

1. Lab objectives

- Interpret entities, attributes, primary keys, foreign keys, and relationships in an ER diagram.
- Translate the supplied Library_ER.drawio diagram into relational tables in MySQL Workbench.
- Use DDL (CREATE, ALTER, DROP) to define and change database structure.
- Use DML (INSERT, SELECT, UPDATE, DELETE) to add, retrieve, modify, and remove records.
- Test referential integrity and explain why related records must be inserted and deleted in a sensible order.

2. Starting files and workflow

1. Open the supplied Library_ER.drawio in diagrams.net (draw.io). Inspect the entities, attributes, key labels, and relationship cardinalities. Do not replace the supplied design with a different scenario.
2. Open MySQL Workbench and connect to your local/class MySQL server.
3. Create a new SQL tab. Save your work regularly as Library_Lab_StudentNumber.sql.
4. Write and execute the SQL statements in the sections below. Run statements in small groups and read the Action Output/messages after each execution.
5. Use the Schemas panel refresh button to see the database and tables. Expand Tables to inspect the structure.

Important: draw.io is a diagram editor, not a database import format. Students should use the diagram as the specification and implement it by writing SQL in Workbench. Do not assume opening the .drawio file creates tables.

3. Read the Library ER diagram

The supplied diagram contains these entities and attributes. PK = primary key, FK = foreign key, UQ = unique constraint.

Entity / table	Attributes from the diagram	Key / relationship notes
Author	AuthorID INT; FirstName VARCHAR(50); LastName VARCHAR(50); Country VARCHAR(50)	PK AuthorID
Category	CategoryID INT; CategoryName VARCHAR(50); Description VARCHAR(255)	PK CategoryID
Book	BookID INT; CategoryID INT; Title VARCHAR(200); ISBN VARCHAR(20); PublicationYear INT; Copies INT	PK BookID; FK CategoryID → Category
BookAuthor	BookID INT; AuthorID INT	Composite PK (BookID, AuthorID); both are FKs
Member	MemberID INT; MemberName VARCHAR(100); Email VARCHAR(100); Phone VARCHAR(20); JoinDate DATE	PK MemberID
MembershipCard	CardID INT; MemberID INT; CardNumber VARCHAR(20); IssueDate DATE; ExpiryDate DATE	PK CardID; MemberID is FK and UNIQUE (one card per member)
Loan	LoanID INT; MemberID INT; BookID INT; LoanDate DATE; DueDate DATE; ReturnDate DATE NULL	PK LoanID; FKs to Member and Book
Reservation	ReservationID INT; MemberID INT; BookID INT; ReservationDate DATE; Status VARCHAR(10)	PK ReservationID; FKs to Member and Book

Relationships:

Category 1:M Book;

Book M:N Author through BookAuthor;

Member 1:M Loan;

Book 1:M Loan;

Member 1:M Reservation;

Book 1:M Reservation;

Member 1:1 MembershipCard.

Loan and Reservation are also associative/transaction tables linking Member and Book.

Design note: the diagram labels the student's number as ending in 5. You can use the same example or update it based on your student number.

4. DDL: Data Definition Language

DDL statements define or change database objects such as schemas and tables. They change the structure, not the individual rows of ordinary data.

```
-- CREATE DATABASE: create a schema for this lab
CREATE DATABASE IF NOT EXISTS LibraryDB;
USE LibraryDB;
```

Explanation: IF NOT EXISTS prevents an error if the schema already exists. USE selects the schema where subsequent unqualified table names will be created.

4.1 Create tables in dependency order

Create parent tables before tables that reference them. The following example implements the entities shown in the diagram. Run it once in a clean LibraryDB schema; if you already created these tables, do not blindly rerun CREATE TABLE.

```
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL,
    Description VARCHAR(255)
);

CREATE TABLE Author (
    AuthorID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Country VARCHAR(50)
);

CREATE TABLE Member (
    MemberID INT PRIMARY KEY,
    MemberName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    Phone VARCHAR(20),
    JoinDate DATE NOT NULL
);
```

```
CREATE TABLE Book (
    BookID INT PRIMARY KEY,
    CategoryID INT NOT NULL,
    Title VARCHAR(200) NOT NULL,
    ISBN VARCHAR(20),
    PublicationYear INT,
    Copies INT NOT NULL DEFAULT 1,
    CONSTRAINT fk_book_category
        FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE BookAuthor (
    BookID INT NOT NULL,
    AuthorID INT NOT NULL,
    PRIMARY KEY (BookID, AuthorID),
    FOREIGN KEY (BookID) REFERENCES Book(BookID),
    FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID)
);

CREATE TABLE MembershipCard (
    CardID INT PRIMARY KEY,
    MemberID INT NOT NULL UNIQUE,
    CardNumber VARCHAR(20) NOT NULL UNIQUE,
    IssueDate DATE NOT NULL,
    ExpiryDate DATE NOT NULL,
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);

CREATE TABLE Loan (
    LoanID INT PRIMARY KEY,
    MemberID INT NOT NULL,
    BookID INT NOT NULL,
    LoanDate DATE NOT NULL,
    DueDate DATE NOT NULL,
    ReturnDate DATE NULL,
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (BookID) REFERENCES Book(BookID)
);

CREATE TABLE Reservation (
    ReservationID INT PRIMARY KEY,
    MemberID INT NOT NULL,
    BookID INT NOT NULL,
    ReservationDate DATE NOT NULL,
    Status VARCHAR(10) NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (BookID) REFERENCES Book(BookID)
);
```

Why these constraints? PRIMARY KEY uniquely identifies each row. FOREIGN KEY prevents references to missing parent rows. NOT NULL requires a value. UNIQUE prevents duplicate values. DEFAULT supplies a value when the column is omitted. ReturnDate allows NULL because a book may not yet have been returned. BookAuthor uses a composite primary key to prevent the same author-book pair being inserted twice.

4.2 ALTER TABLE: change an existing table

```
-- Example: add a shelf location to Book
ALTER TABLE Book
ADD COLUMN ShelfLocation VARCHAR(20);

-- Inspect the updated structure
DESCRIBE Book;
```

ALTER TABLE changes the structure of a table. Execute the ADD COLUMN statement once; running it again will fail because the column already exists.

4.3 DROP TABLE: remove a table (demonstration only)

```
-- DANGER: permanently removes the table and its rows.
-- Do not run this against your completed lab database.
-- DROP TABLE Reservation;
```

DROP TABLE is DDL. It removes the table object, not merely its rows. Foreign-key dependencies can prevent a table from being dropped until dependent tables are handled.

4.4 Inspect your schema

```
SHOW TABLES;
DESCRIBE Book;
SHOW CREATE TABLE Book;
```

5. DML: Data Manipulation Language

DML changes or retrieves the records stored inside tables. The common commands in this lab are INSERT, SELECT, UPDATE, and DELETE.

5.1 INSERT: add sample records

Insert parent records first, then records in child tables. These are fictional example records.

```
INSERT INTO Category (CategoryID, CategoryName, Description)
VALUES
(1, 'Database', 'Database and data management books'),
(2, 'Fiction', 'Fiction and literature');

INSERT INTO Author (AuthorID, FirstName, LastName, Country)
VALUES
(1, 'Amina', 'Hassan', 'Egypt'),
(2, 'James', 'Rivera', 'USA');

INSERT INTO Member (MemberID, MemberName, Email, Phone, JoinDate)
VALUES
(101, 'Sara Ali', 'sara@example.edu', '555-0101', '2026-09-01'),
(102, 'Omar Demir', 'omar@example.edu', '555-0102', '2026-09-03');

INSERT INTO Book
(BookID, CategoryID, Title, ISBN, PublicationYear, Copies)
VALUES
(201, 1, 'Learning SQL', '9780000000001', 2024, 4),
(202, 2, 'The Silent Library', '9780000000002', 2022, 2);

INSERT INTO BookAuthor (BookID, AuthorID)
VALUES (201, 1), (202, 2);

INSERT INTO MembershipCard
(CardID, MemberID, CardNumber, IssueDate, ExpiryDate)
VALUES
(301, 101, 'CARD-101', '2026-09-01', '2027-09-01'),
(302, 102, 'CARD-102', '2026-09-03', '2027-09-03');

INSERT INTO Loan
(LoanID, MemberID, BookID, LoanDate, DueDate, ReturnDate)
```

```
VALUES
(401, 101, 201, '2026-10-01', '2026-10-15', NULL);

INSERT INTO Reservation
(ReservationID, MemberID, BookID, ReservationDate, Status)
VALUES
(501, 102, 201, '2026-10-02', 'Pending');
```

INSERT INTO table (columns...) VALUES (...); adds a row. Listing columns explicitly makes the statement easier to read and safer if the table structure later changes. Use single quotes around text and dates. Use NULL without quotes for a missing return date.

5.2 SELECT: read data

```
-- All columns and rows
SELECT * FROM Book;

-- Only selected columns
SELECT Title, PublicationYear, Copies
FROM Book;

-- Filter rows
SELECT Title, Copies
FROM Book
WHERE Copies >= 3;

-- Sort results
SELECT Title, PublicationYear
FROM Book
ORDER BY PublicationYear DESC;
```

SELECT does not change stored rows. WHERE filters the rows returned; ORDER BY sorts the result. DESC means descending; ASC means ascending.

5.3 JOIN: query relationships

```
-- Show each book with its category
SELECT b.BookID, b.Title, c.CategoryName
FROM Book AS b
JOIN Category AS c
  ON b.CategoryID = c.CategoryID;

-- Show loans with member and book names
SELECT l.LoanID, m.MemberName, b.Title,
       l.LoanDate, l.DueDate, l.ReturnDate
FROM Loan AS l
JOIN Member AS m ON l.MemberID = m.MemberID
JOIN Book AS b ON l.BookID = b.BookID;

-- Show authors of each book
SELECT b.Title, a.FirstName, a.LastName
FROM Book AS b
JOIN BookAuthor AS ba ON b.BookID = ba.BookID
JOIN Author AS a ON ba.AuthorID = a.AuthorID;
```

A JOIN combines related rows using matching key values. Table aliases (b, c, l, m, a, ba) shorten names and make the query easier to read.

5.4 UPDATE: modify records

```
-- Update a book's number of copies
UPDATE Book
```

```
SET Copies = 5
WHERE BookID = 201;

-- Mark a loan as returned
UPDATE Loan
SET ReturnDate = '2026-10-10'
WHERE LoanID = 401;

-- Verify the changes
SELECT * FROM Book WHERE BookID = 201;
SELECT * FROM Loan WHERE LoanID = 401;
```

UPDATE changes existing values. Always check the WHERE condition before running it: without WHERE, every row in the table may be updated.

5.5 DELETE: remove records

```
-- Remove one reservation using its primary key
DELETE FROM Reservation
WHERE ReservationID = 501;

-- Verify that it was removed
SELECT * FROM Reservation
WHERE ReservationID = 501;
```

DELETE removes rows, not the table structure. Without WHERE, every row is removed. A row referenced by a foreign key may not be deletable until dependent records are removed or updated.

6. DDL vs DML at a glance

Command	Category	Effect
CREATE TABLE	DDL	Creates a table structure
ALTER TABLE	DDL	Changes a table structure
DROP TABLE	DDL	Removes a table and its data
INSERT	DML	Adds rows
SELECT	DML	Reads rows; does not modify data
UPDATE	DML	Changes existing rows
DELETE	DML	Removes selected rows

7. Student tasks (complete independently)

6. Create LibraryDB and all eight tables represented in the diagram. Include every listed column and implement the primary keys, foreign keys, and the unique MemberID constraint in MembershipCard.
7. Add a ShelfLocation column to Book using ALTER TABLE. Choose a suitable value for each book and update the two sample books.
8. Insert at least 3 categories, 4 authors, 5 books, 5 members, 5 membership cards, 4 BookAuthor pairs, 4 loans, and 3 reservations. Use valid foreign-key values.
9. Write a SELECT query that lists all books with their category names.
10. Write a SELECT query that lists every member who has a loan, with the book title and due date.
11. Write a query to list books that have 3 or more copies, sorted by title.
12. Write a query to show each book and its author(s), including the junction table BookAuthor.
13. Update one reservation status to 'Fulfilled' or 'Cancelled' and show the result.
14. Set ReturnDate for one loan, then query all loans that have not yet been returned (ReturnDate IS NULL).
15. Delete one reservation by ReservationID and show that it no longer appears.
16. Demonstrate referential integrity: try inserting a Book with a CategoryID that does not exist. Record the error, then correct it by using a valid CategoryID. Do not leave invalid data in the final database.

17. Use DESCRIBE and SHOW CREATE TABLE to demonstrate the structure and constraints of at least two tables.

8. Questions for your lab report

- 1. What is the difference between a primary key and a foreign key?
- 2. Why does BookAuthor have a composite primary key?
- 3. How does UNIQUE on MembershipCard.MemberID help implement the 1:1 relationship?
- 4. Why must Category rows exist before inserting books that reference them?
- 5. Explain the difference between DELETE FROM Reservation WHERE ReservationID = 501 and DROP TABLE Reservation.
- 6. What happens if UPDATE Book SET Copies = 0 is run without a WHERE clause? (Do not run this statement.)
- 7. Why is ReturnDate allowed to be NULL?
- 8. In your own words, explain how JOIN uses primary/foreign keys to combine records.

9. Submission checklist

- ☐ SQL script saved as Library_Lab_StudentNumber.sql and runs from a clean schema.
- ☐ All eight tables, key constraints, and relationships are implemented.
- ☐ Required sample data and task queries are included as SQL comments/sections.
- ☐ Include screenshots of SHOW TABLES, DESCRIBE Book, the book/category JOIN, and the loan/member/book JOIN.
- ☐ Answer the lab-report questions in a separate document or at the end of the SQL script.
- ☐ Do not submit passwords, connection credentials, or private personal information.

10. Troubleshooting

Error 1046: No database selected: Run USE LibraryDB; before creating or querying tables, or qualify the table name with the schema.

Error 1452: foreign key constraint fails: The referenced parent key does not exist. Insert the parent record first or correct the FK value.

Error 1062: duplicate entry: A primary key or unique value already exists. Choose a new ID or unique value.

Table already exists: You may have run the script before. Inspect the existing table before changing or deleting anything; avoid dropping tables that contain work.

UPDATE/DELETE changed too many rows: Use SELECT with the same WHERE condition first, then run the change with a specific primary key in WHERE.